Atmospheric: how the code works and how it was built

OPSC6312 final PoE - Group 24 - Oluga Neluvhalani (ST10443034) and Luthando Mndawe (ST10446457)
Repository: github.com/ST10443034/atmospheric

1. What the app is

Atmospheric is an offline-first Android weather app written in Kotlin. It shows live weather from the open Open-Meteo REST API and, when the network fails, falls back to the last reading saved on the phone and labels it as CACHED. It uses classic Android Views (XML layouts with view binding), not Jetpack Compose.

Area	Technology
Architecture	MVVM, single Activity with Fragments, Jetpack Navigation and a bottom navigation bar
REST API	Retrofit and Gson calling Open-Meteo
Offline cache	Room database, with a SyncQueue table for pending work
Settings	Jetpack DataStore (theme, units, language, notifications)
Background work	WorkManager hourly job with a network constraint and exponential backoff
Accounts	Firebase Authentication: email and password, plus Google Sign-In
Cloud data	Cloud Firestore document users/{uid}; profile photo stored as a Base64 JPEG
Push	Firebase Cloud Messaging, topic weather_alerts
Languages	English, isiZulu and Afrikaans, switched inside the app
Build and CI	Gradle Kotlin DSL, GitHub Actions, signed release APK

2. How we built it, in stages

- **Part 1 - design.** We planned the five screens (login, dashboard, forecast, sync and offline, settings), the offline-first idea and the data model.
- **Part 2 - prototype.** We built the Retrofit client, the Room cache, DataStore settings, the screens and navigation, and Firebase email login. The lecturer then pointed out three gaps: a placeholder 2 + 2 unit test, units that were stored but never applied, and a notification switch that did nothing.
- **Part 3 - final.** We fixed those three points and added Google sign-in, Firestore profile and settings sync, the photo blob, push and local alerts, the isiZulu and Afrikaans strings, real unit tests, CI, and release signing.

3. Data flow

The UI never calls the network. Data moves one way: **Open-Meteo, Retrofit, WeatherRepository, Room, ViewModel, Fragment**. The repository is the single source of truth. It tries a live fetch, writes the result to Room, and if the fetch fails it returns the cached row with a CACHED flag. The ViewModel exposes the result as a StateFlow and the Fragment only draws it. This keeps screens simple and makes the offline behaviour the same everywhere.

4. The main pieces of code

File or class	What it does and why
`data/repository/WeatherRepository.kt`	Fetches the forecast, maps it to a Room entity (the public extension CurrentWeather.toEntity), and falls back to the cache on failure. The background worker reuses the same mapping.
`domain/model/UnitSystem.kt`	An enum with METRIC and IMPERIAL. Data is always stored in metric and converted only for display (for example 23 C shows as 73 F and 12 km/h as 7 mph). Screens collect the units setting from DataStore and re-render, so a change updates every screen. This fixed the hard-coded units.
`domain/model/AlertRules.kt`	Pure Kotlin that picks the most severe alert from the current weather: thunderstorm (code 95 to 99), heat (35 C or more), rain (80 percent chance or more), wind (50 km/h or more), in that priority order.
`work/WeatherSyncWorker.kt and SyncScheduler.kt`	An hourly CoroutineWorker. It refreshes the Room cache and, if the user turned notifications on, posts a local alert whose text follows the chosen units. It retries up to three times. It is scheduled from the Application class.
`notifications/ (NotificationHelper, PushManager, AtmosphericMessagingService)`	Creates the notification channel, checks the Android 13 permission, subscribes or unsubscribes the device from the weather_alerts topic, and handles incoming FCM messages while respecting the Settings switch.
`data/repository/AuthRepository.kt and ui/login/`	Email register and login, and Google sign-in (the Google ID token is exchanged for a Firebase credential). Passwords are hashed by Firebase and are never stored by the app.
`data/repository/UserProfileRepository.kt`	Reads and writes the Firestore document users/{uid}: name, photo, units, language, theme, notifications and the FCM token. Writes are merge sets and are not awaited, so Firestore's offline queue uploads them later.
`util/ImageCodec.kt`	Scales a chosen photo down to 256 px, compresses it to JPEG and converts it to Base64, and the reverse. This is the blob storage, kept in Firestore so no paid storage plan is needed.
`ui/settings/SettingsViewModel.kt and SettingsFragment.kt`	Ties settings to DataStore, Firestore and push: the notification switch asks for permission, updates DataStore, subscribes to the topic and saves the choice online. The photo picker and profile card live here.

5. What is stored online

- **Authentication:** email, sign-in provider and user ID. Passwords are salted and hashed by Firebase.
- **Firestore, users/{uid}:** displayName, photoBase64, units, language, theme, notificationsEnabled, fcmToken, email and updatedAt. The security rules let only the signed-in owner read or write their own document and deny everything else.
- **Open-Meteo:** read-only public data. Nothing about the user is stored there.

6. Testing, CI and release

- **Unit tests** (app/src/test): UnitSystemTest, AlertRulesTest and ImageCodecTest. They replaced the placeholder example test.
- **GitHub Actions CI** (blank.yml): on each push and pull request it builds the app, runs Lint and the unit tests, and uploads the debug APK and reports.
- **Release** (release.yml): a v* tag builds a signed APK using a keystore and passwords kept in GitHub secrets. Locally the release build reads a git-ignored keystore.properties file, so no keys are in the repository.
- **Logging:** Log calls around sign-in, sync, notifications and profile saving help trace problems.

7. Limits and AI use

Known limits: the location is fixed to Johannesburg, only the current reading is cached (hourly and daily lists are empty offline until a live fetch), and the isiZulu and Afrikaans strings are machine-assisted and still need review by a native speaker.

We used Claude (Anthropic, through Claude Code) as an AI assistant for parts of the Part 3 code, the tests, the build and CI setup and the documentation. This is declared in docs/AI_USAGE.md.